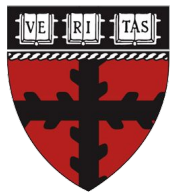
Lecture 11: Data Labeling, Versioning and Monitoring

AC215/E115

Pavlos Protopapas
SEAS/Harvard



Outline

1. Data Labeling
2. Data Versioning
3. Data Monitoring

Outline

1. **Data Labeling**
2. Data Versioning
3. Data Monitoring

Why Data Labeling?

Model Learning

- Converts raw data into examples that supervised ML models can learn from.

Ground Truth

- Provides the correct answers used to train and evaluate the model.

Model Quality

- Accurate, consistent labels improve model performance; incorrect labels teach the model incorrect patterns.

Real-World Context

- Human annotators capture meaning and context that may be difficult to identify automatically.

Continuous Improvement:

- Labeling model errors and difficult edge cases helps create better versions of the training dataset.

Tutorial: Data Labeling

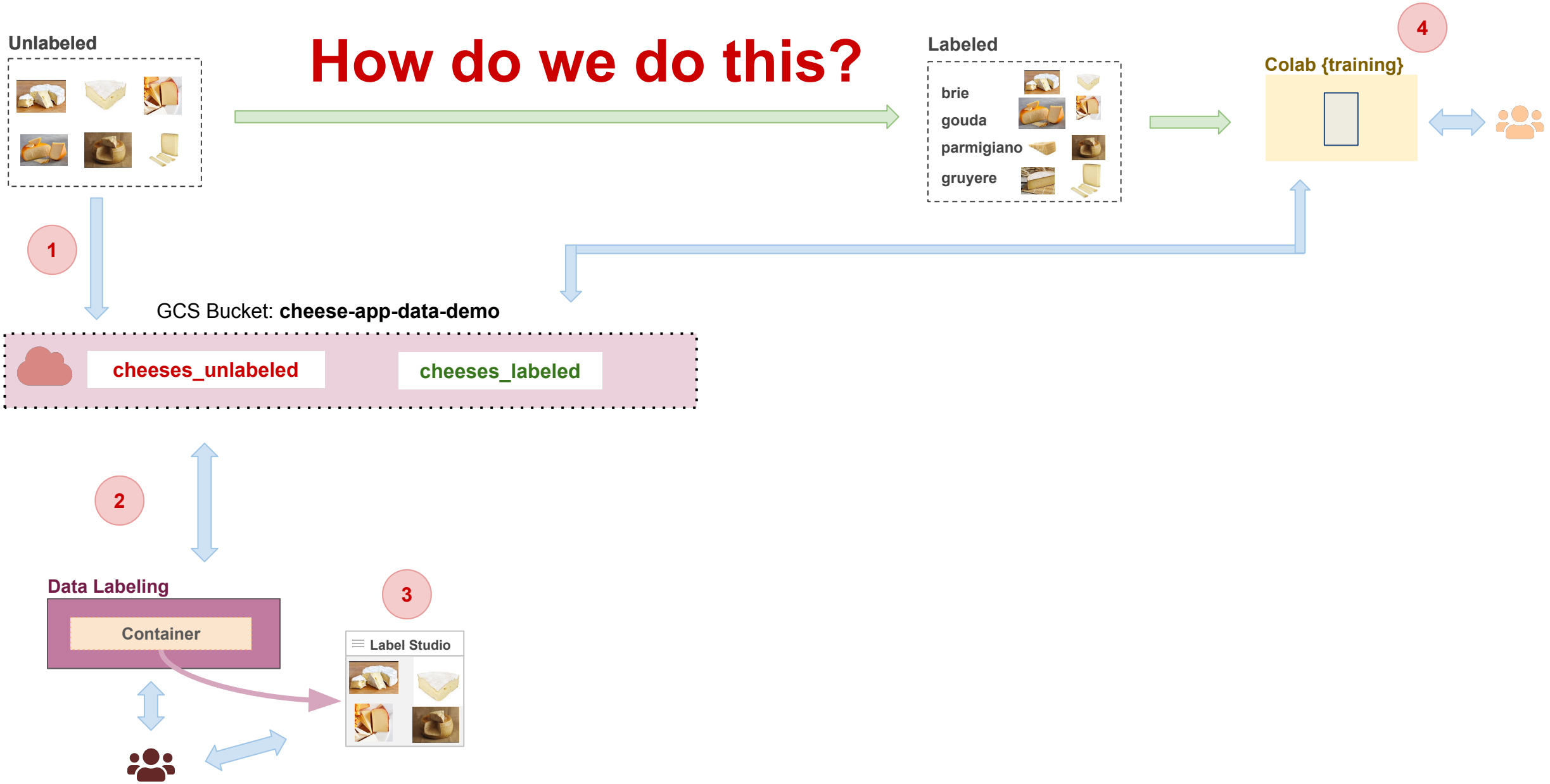
In this tutorial, we'll learn how to label data efficiently.

Our example dataset contains images of cheeses: Brie, Gruyère, Gouda, and Parmigiano.

We'll start from unlabeled web images and use **Label Studio** to annotate them.



Cheese App Data Pipeline



Tutorial: Why Label Studio?

- Labeling large datasets manually can be slow and inconsistent.
- Label Studio streamlines the process with an easy-to-use interface.
- To keep our data private, we'll run Label Studio locally inside a Docker container — nothing gets uploaded externally.

Recap Concepts Before Labeling

Before we jump into labeling, let's revisit three familiar concepts:

- Docker Network
- Cloud Storage and Credentials
- Docker Compose

Think you remember what each one does?

Recap Concepts Before Labeling

Before we jump into labeling, let's revisit three familiar concepts:

- **Docker Network**
- Cloud Storage and Credentials
- Docker Compose

Think you remember what each one does?

Recap Concepts Before Labeling

Before we jump into labeling, let's revisit three familiar concepts:

- **Docker Network:** Allows containers to communicate with each other.
- Cloud Storage and Credentials
- Docker Compose

Recap Concepts Before Labeling

Before we jump into labeling, let's revisit three familiar concepts:

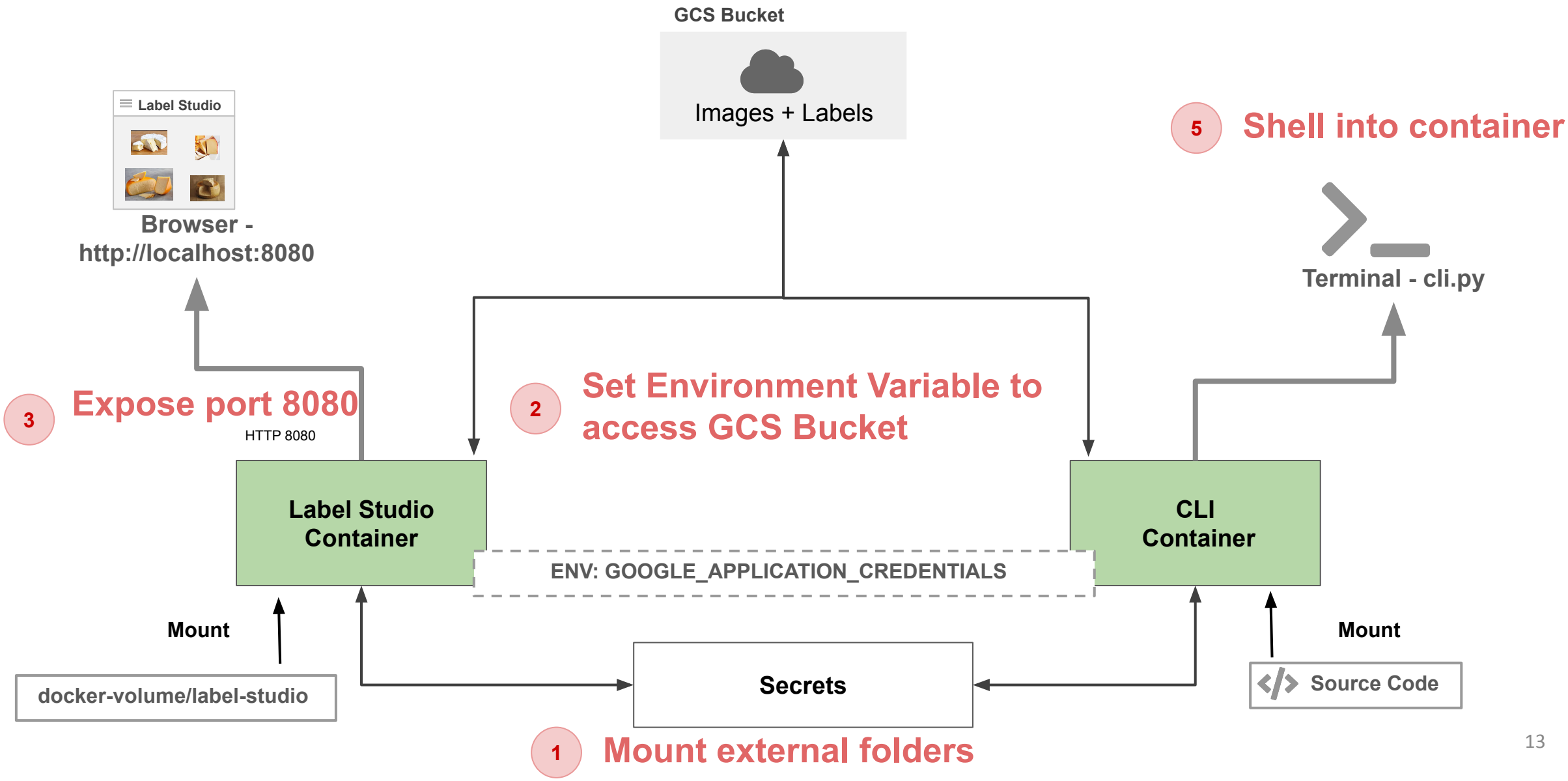
- **Docker Network:** Allows containers to communicate with each other.
- **Cloud Storage and Credentials:** Stores data online and securely controls access to it.
- Docker Compose

Recap Concepts Before Labeling

Before we jump into labeling, let's revisit three familiar concepts:

- **Docker Network:** Allows containers to communicate with each other.
- **Cloud Storage and Credentials:** Stores data online and securely controls access to it.
- **Docker Compose:** Defines and runs multiple related containers together.

Tutorial: Data Labeling: Label Studio + CLI



Tutorial: Data Labeling

Steps to create a Data Pipeline to use unlabeled images and create a processes to label the dataset:

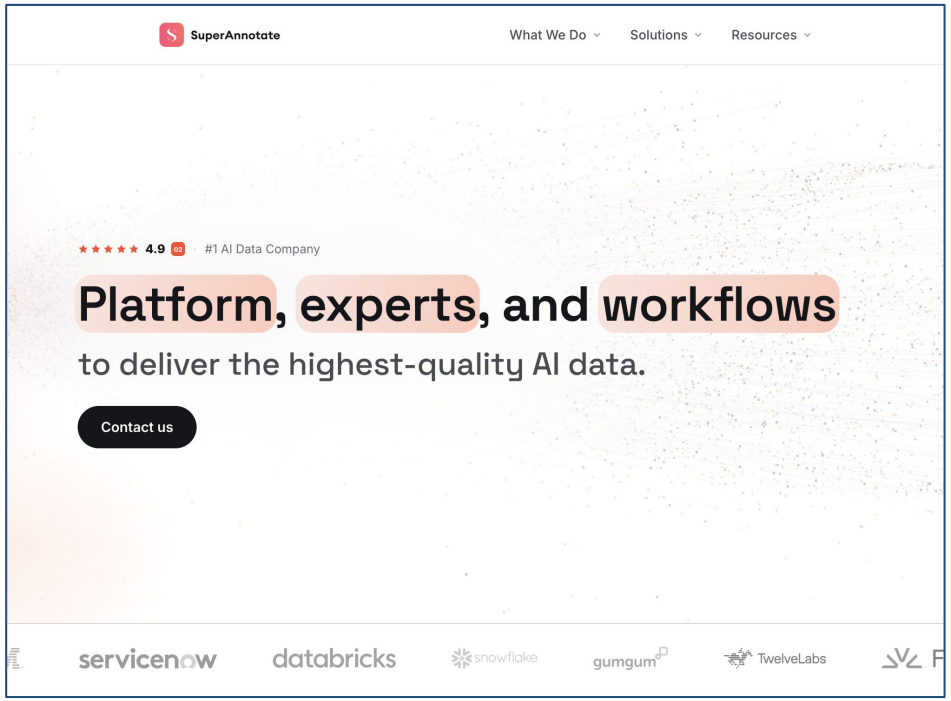
- Create a GCS bucket to store all data.
- Run Data Labeling Container.
- For detailed instructions, please refer to the following link
  - Data Labeling. (https://github.com/dlops-io/data-labeling)



Data Labeling (Continued)

Other data labeling tools that are widely used:

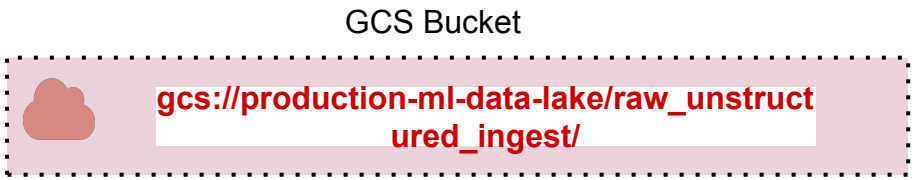
- **SuperAnnotate**
  - for multimodal labeling: image, video, audio, RLHF, SFT



Data Labeling (Continued)

Other data labeling tools that are widely used:

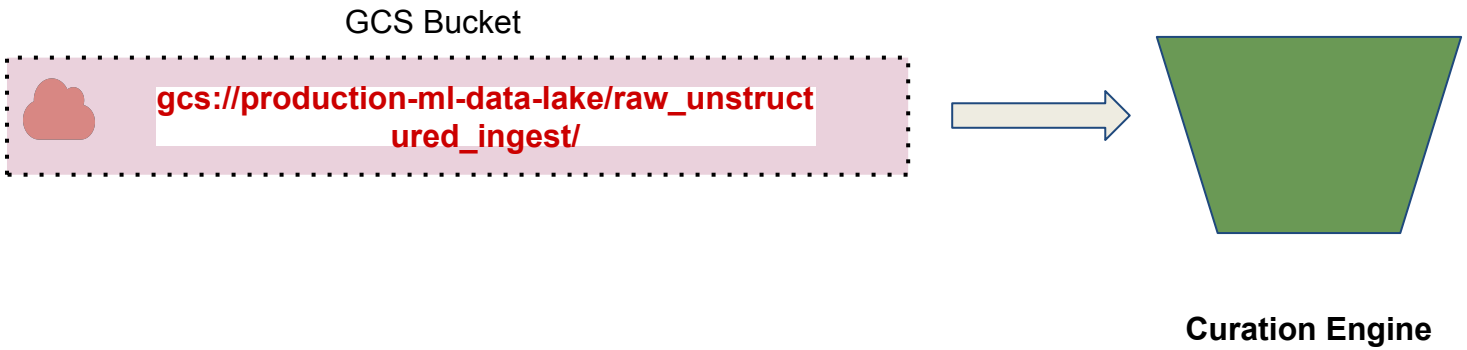
- **SuperAnnotate**
  - for multimodal labeling: image, video, audio, RLHF, SFT



Data Labeling (Continued)

Other data labeling tools that are widely used:

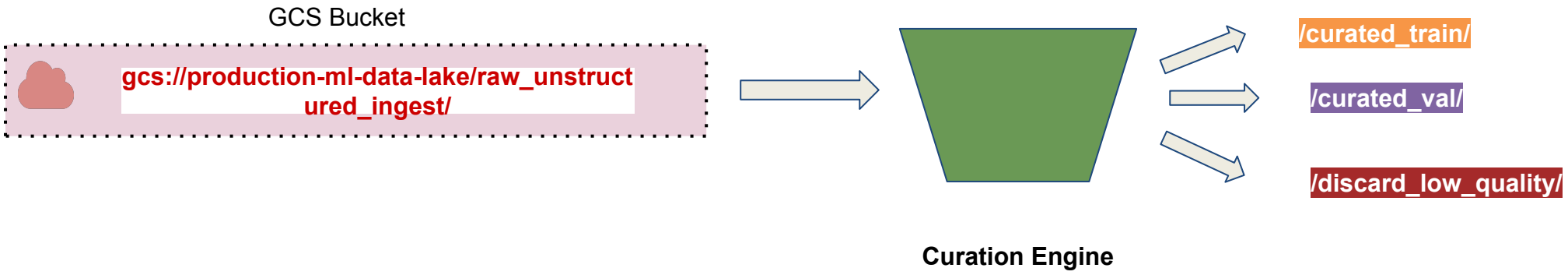
- **SuperAnnotate**
  - for multimodal labeling: image, video, audio, RLHF, SFT



Data Labeling (Continued)

Other data labeling tools that are widely used:

- **SuperAnnotate**
  - for multimodal labeling: image, video, audio, RLHF, SFT



Outline

1. Data Labeling
2. **Data Versioning**
3. Data Monitoring

Why Data Versioning?

Keep Track:

- Monitor data changes and stay organized.

Protection:

- Backup data and restore earlier versions if needed.

Compliance:

- Meet regulations by tracking changes, making audits simple.

Collaboration:

- Let multiple users work together smoothly.

Efficiency:

- Reduce unnecessary duplication through snapshots, content-based storage, or other platform-specific methods.

Key Vocabulary

Concepts

1. **Data lake**: Cloud storage containing large amounts of raw or processed data.
2. **Table format**: A structure such as Delta Lake or Iceberg that manages tables, metadata, and snapshots.
3. **Time travel:** The ability to access an earlier version of data.

Tools

1. **DVC**: A Git-based tool for versioning files, datasets, and ML experiments.
2. **lakeFS**: A version-control layer for large object-storage data lakes.

Approaches to Data Versioning

File-based versioning: Tracks files using hashes and metadata **e.g.** DVC

Object-store versioning: Provides Git-like branches, commits, and rollbacks over cloud data lakes **e.g.** lakeFS

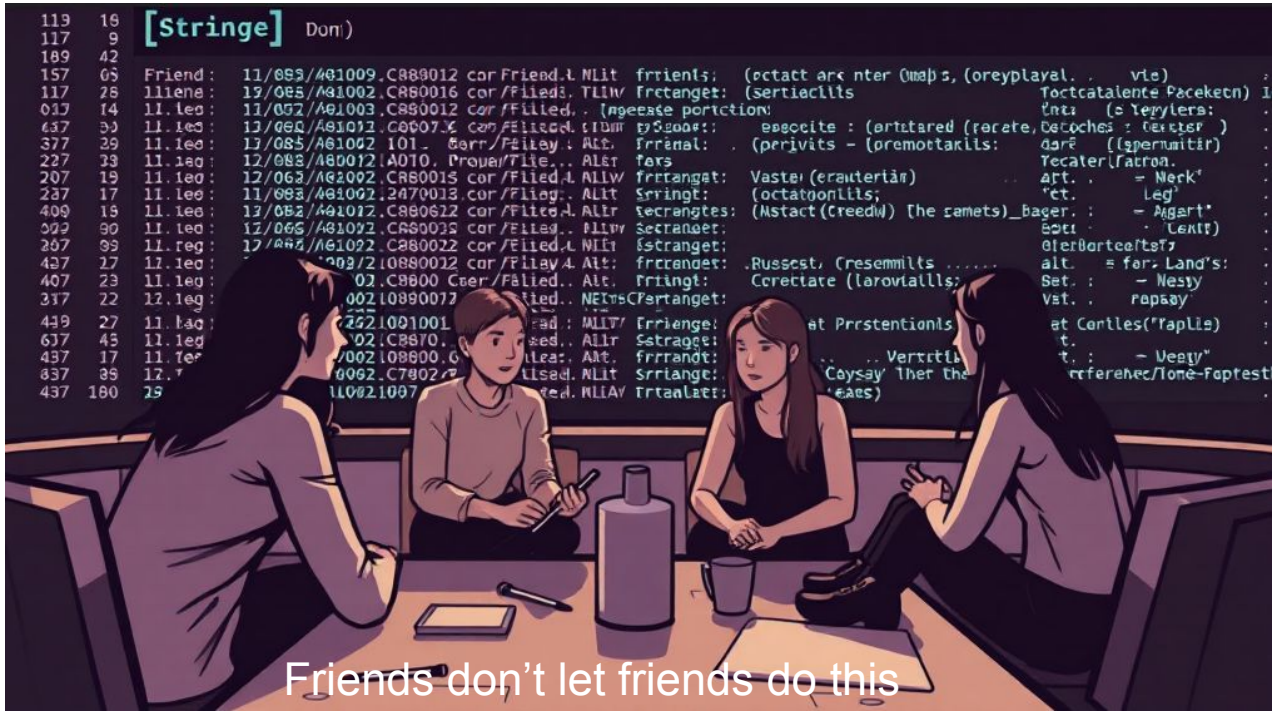
Table-based versioning: Uses transaction logs, metadata, and snapshots for structured tables **e.g.** Delta Lake and Iceberg

Full snapshots: Stores a complete dataset state at selected points when appropriate.

And … “wait for it”

Approaches to Data Versioning

```
(base) pavlosprotopapas@Pavloss-Mac-mini PINNS-TransferLearning-Paper % ls
ICML-subm-DM-v1.tex                         ICML-subm-final-final-v2.tex
ICML-subm-PP-v1.1-IB-PO-v2.tex              ICML-subm-final-final-v3-final.tex
ICML-subm-PP-v1.1-IB.tex                    ICML-subm-final-final-v3.tex
ICML-subm-PP-v1.1.tex                       ICML-subm-final-final.tex
ICML-subm-PP.tex                            ICML-subm-final-v2.tex
ICML-subm-final-final-v1.tex                ICML-subm-final.tex
(base) pavlosprotopapas@Pavloss-Mac-mini PINNS-TransferLearning-Paper %
```



Choosing a Data Versioning Approach

Tool	Best suited for	How it versions data
DVC	Reproducible ML experiments and file-based datasets	Content-addressed files referenced through Git
lakeFS	Large object-storage data lakes	Git-like commits, branches, and snapshots
Delta Lake / Iceberg	Structured lakehouse tables	Table metadata, transaction logs, and snapshots

Tutorial: Data Versioning

Steps to create a Data Pipeline to **version a dataset**:

- Run Data Versioning Container.
- Test data versions from Colab.
- For detailed instructions, please refer to the following link
  - Data Versioning (https://github.com/dlops-io/data-versioning)
  - Test Data Version Notebook (https://colab.research.google.com/drive/1RRQ1SlHq5lKK76R8LoQdi5LjCnND3jTq?authuser=1)



Tutorial: Data Versioning

Before we proceed, we need to familiarize ourselves with some new concepts:

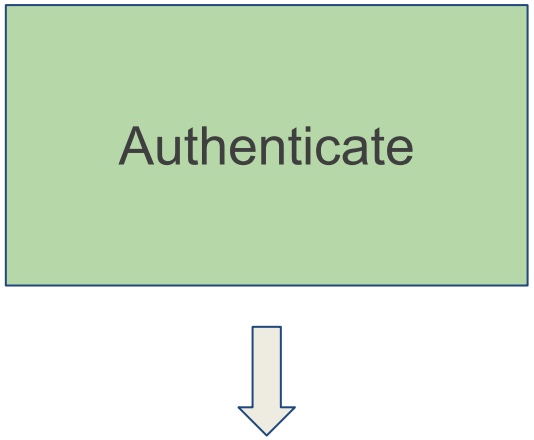
- Mounting disks from GCS to a container
- Entrypoint

Mounting disks from GCS to a container

gcsfuse is an open-source tool by Google which lets a container access a Google Cloud Storage (GCS) bucket from a local folder.

Mounting disks from GCS to a container

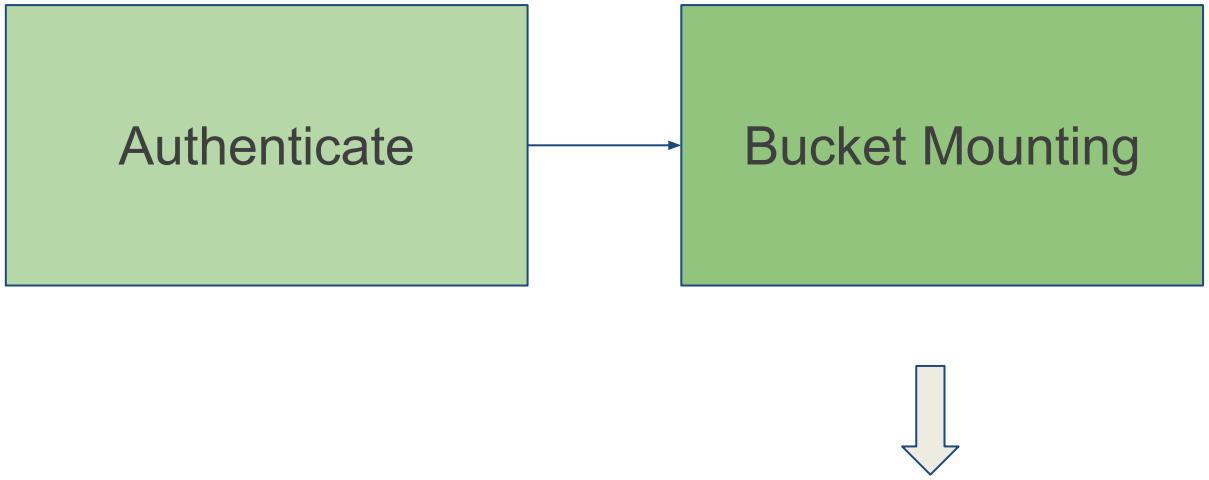
How the gcsfuse mounting works?



The service-account credentials give gcsfuse permission to access GCS

Mounting disks from GCS to a container

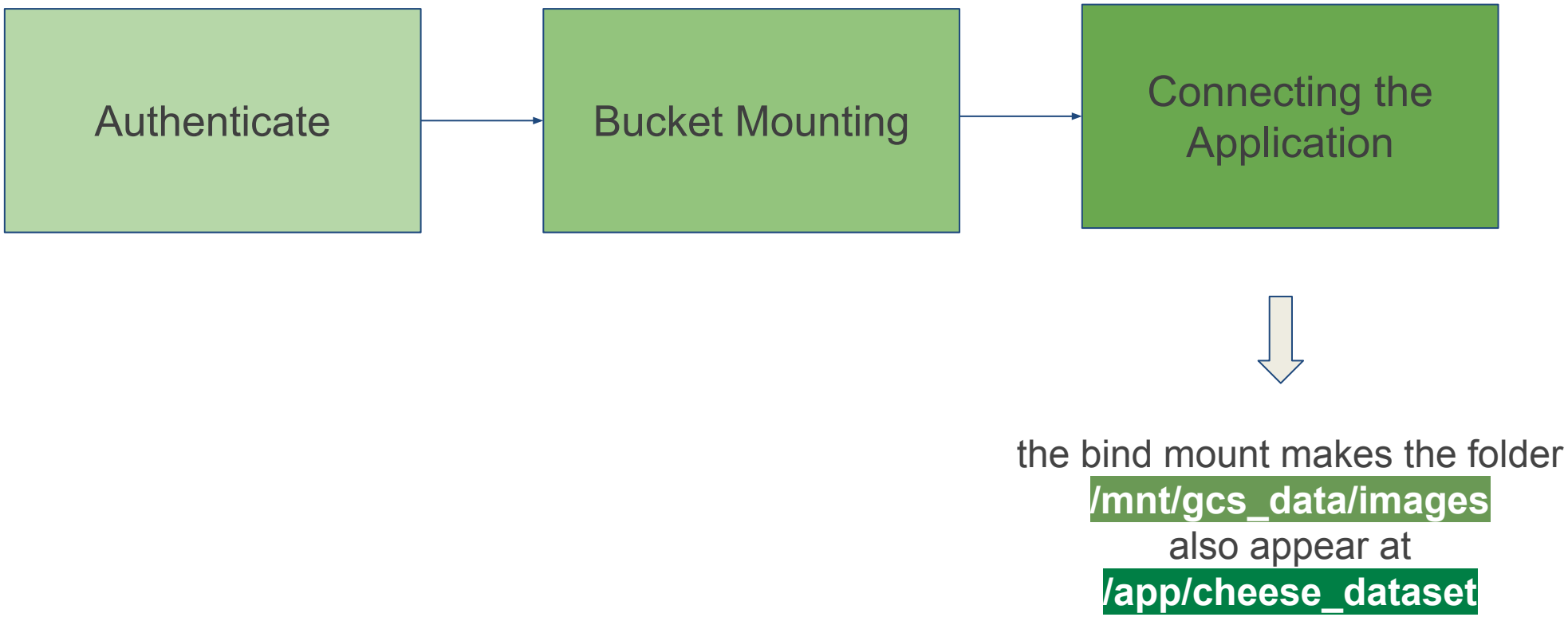
How the gcsfuse mounting works?



gcsfuse makes the files stored in the GCS bucket appear inside the container as a regular folder called **/mnt/gcs_data**

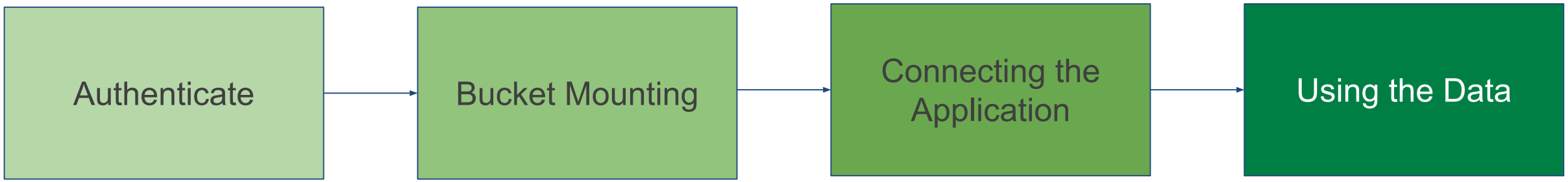
Mounting disks from GCS to a container

How the gcsfuse mounting works?



Mounting disks from GCS to a container

How the gcsfuse mounting works?

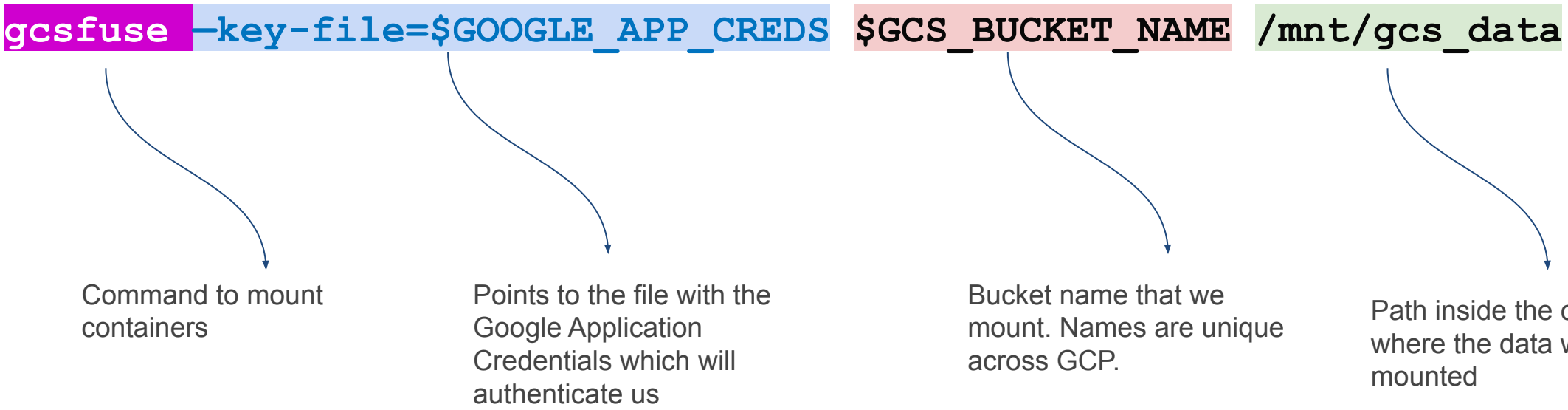


The application and DVC work with **/app/cheese_dataset** like a local folder, while reads and writes go to the GCS bucket.

Mounting disks from GCS to a container

In order to use the data on the container, we have to link them. This is done via gcsfuse.

```
gcsfuse --key-file=$GOOGLE_APPLICATION_CREDENTIALS
$GCS_BUCKET_NAME /mnt/gcs_data
```



Entrypoints

In Docker, entrypoints allow us to define a specific application or command that runs automatically when the container starts.
This enables us to automate tasks inside the container, or even make the `docker run` command behave like executing a standalone, containerized program.

Entrypoints

For example, an entrypoint can run a script

```
ENTRYPOINT ["/myapp/start.sh"]
```

or a program with arguments

```
ENTRYPOINT ["python", "cli.py"]
```

In development containers,

```
ENTRYPOINT ["/bin/bash"]
```

is used to open a shell terminal for interactive commands.

Takeaway

There is **no single** industry standard for data versioning tools. The right tool depends on the data type, scale, and platform.



lakeFS (Another Data Versioning Tool)

As of 2025, **lakeFS** acquired DVC

lakeFS provides Git-like version control for datasets **stored in cloud storage**, allowing teams to track changes, test updates separately, and restore an earlier version when needed.

lakeFS vs DVC

Feature	DVC	lakeFS
Target users	Technical engineers	Data Infrastructure
Use Case	Tracking local experiments	Managing massive data lakes
Scaling	Lightweight files	Multi-petabyte object storage (S3/GCS/Azure)

lakeFS vs DVC (Final Remarks)

DVC is a **widely used open-source tool** for teaching reproducibility and managing file-based datasets in ML projects.

Larger production systems may use lakeFS, Delta Lake, Iceberg, or platform-native solutions depending on their architecture.

Outline

1. Data Labeling
2. Data Versioning
3. **Data Monitoring**

Why Data Monitoring?

Data Quality:

Catch broken formatting, missing fields, and schema drift instantly.

Data Freshness:

Monitor pipeline delays to ensure dashboards and AI models use real-time data.

Anomalies and Drift:

Detect sudden spikes or shifts in data distribution before it breaks AI accuracy.

Cost Control:

Track cloud compute and query costs to stop runaway infrastructure bills.

Observability:

Trace end-to-end data lineage to easily find exactly where a pipeline failed.

Approaches to Data Monitoring

Rules-Based: Only specific constraints are tested since the schema and pipeline expectations are predefined.

Anomaly Detection: Baseline metrics are tracked continuously by ML models to automatically flag unusual shifts or drops in data.

All Other Cases: Monitoring is based on tracing end-to-end lineage and metadata, enabling quick root-cause analysis across complex pipelines.

Tools for Data Monitoring

Monte Carlo

- End-to-end data observability using automated ML models.
- Tracks data freshness, volume changes, and schema drift across warehouses.

Datadog

- Combines traditional application tracing with automated data profiling.
- Monitors database performance and anomalies directly alongside core infrastructure.

Soda

- SQL-accessible data testing and observation layer.
- Enables data engineering teams to write and execute programmatic quality checks.



THANK YOU